

MEDS Lab - Module 4

Section 7 Report: Decoders, Encoders & One-Hot Logic

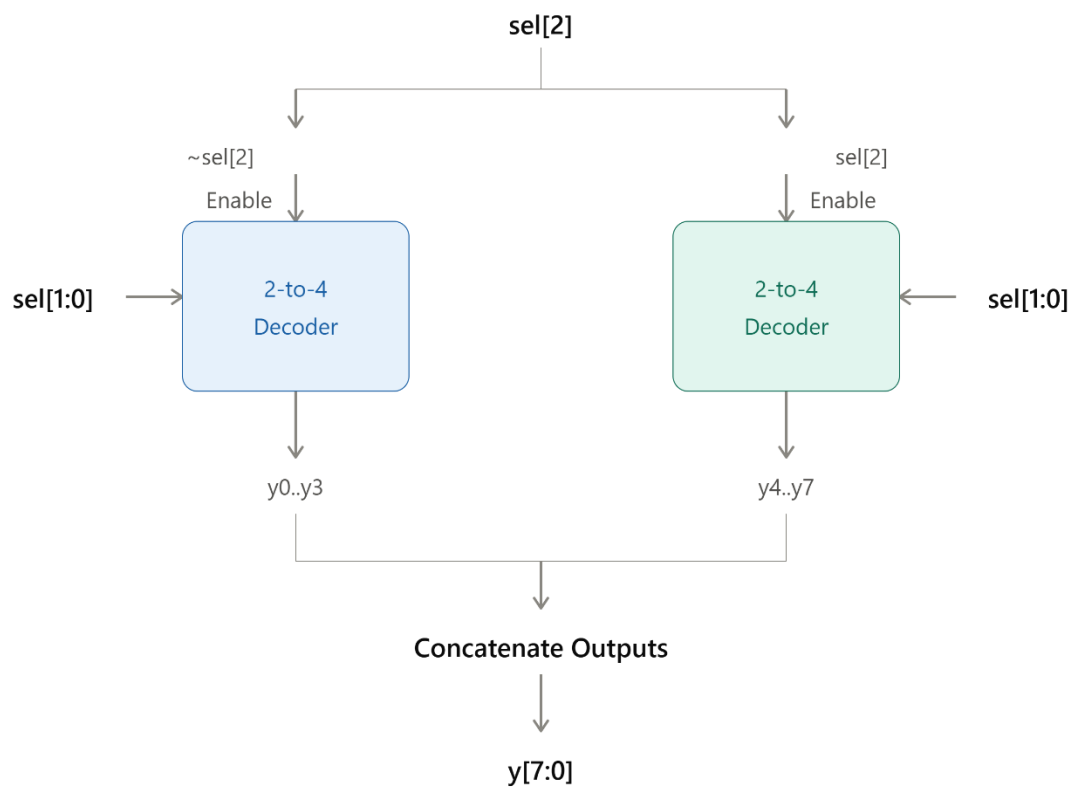
Name: Muhammad Kaleem Faryad

University: UET Lahore

Task 6: 3-to-8 Decoder from 2-to-4 Decoders

1. EDA Playground Link: <https://edaplayground.com/x/SuLM>

2. Block Diagram:



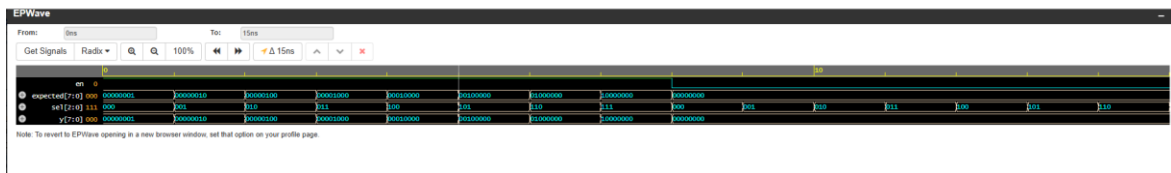
3. Simulation Output Screenshot:

```
testbench.vv
13
14
15 // Expected output
16 logic [7:0] expected;
17
18 // Self-check Task
19

design.vv
6 );
7
8 always_comb begin
9     if(len)
10         y = 4'b0000;
11     else begin
12         case(sel)
13
```

```
Log
Top Level Modules:
tb
TimeScale is 1 ns / 1 ns
Starting vcs inline pass...
1 module and 0 UDP read.
recompiling module tb
rm -f _cuarc*.so _csrc*.so pre_vcsobj_*.so share_vcsobj_*.so
if [ -x ../simv ]; then chmod a-x ../simv; fi
g++ -o ../simv -rdynamic -Wl,-rpath=$ORIGIN/simv.daidir -Wl,-rpath=/apps/vcsmx/vcs/X-2025.06-SP1/linux64/t1b -L/apps/vcsmx/vcs/X-2025.06-SP1/linux64/
../simv up to date
CPU time: .637 seconds to compile + .484 seconds to elab + .561 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 20 12:58 2026
[PASS] en=1 sel=000 output=000000001
[PASS] en=1 sel=001 output=000000010
[PASS] en=1 sel=010 output=000001000
[PASS] en=1 sel=011 output=000010000
[PASS] en=1 sel=100 output=000100000
[PASS] en=1 sel=101 output=001000000
[PASS] en=1 sel=110 output=010000000
[PASS] en=1 sel=111 output=100000000
[PASS] en=0 sel=000 output=000000000
[PASS] en=0 sel=001 output=000000000
[PASS] en=0 sel=010 output=000000000
[PASS] en=0 sel=011 output=000000000
[PASS] en=0 sel=100 output=000000000
[PASS] en=0 sel=101 output=000000000
[PASS] en=0 sel=110 output=000000000
[PASS] en=0 sel=111 output=000000000
$finish called from file "testbench.vv", line 58.
$finish at simulation time 16
V C S   S i m u l a t i o n   R e p o r t
Time: 16 ns
CPU Time: 0.550 seconds; Data structure size: 0.0Mb
Mon Jul 20 12:58:45 2026
Finding VCD file...
./dump.vcd
```

4. Waveform Screenshot:



5. **Explanation:** A decoder converts an n-bit binary input into one active output line. A 3-to-8 decoder activates exactly one of eight outputs according to the three-bit input value. Instead of directly writing the logic, this implementation follows a hierarchical hardware design approach by building the larger decoder from two smaller 2-to-4 decoders. The most significant select bit (sel[2]) decides which decoder is enabled.

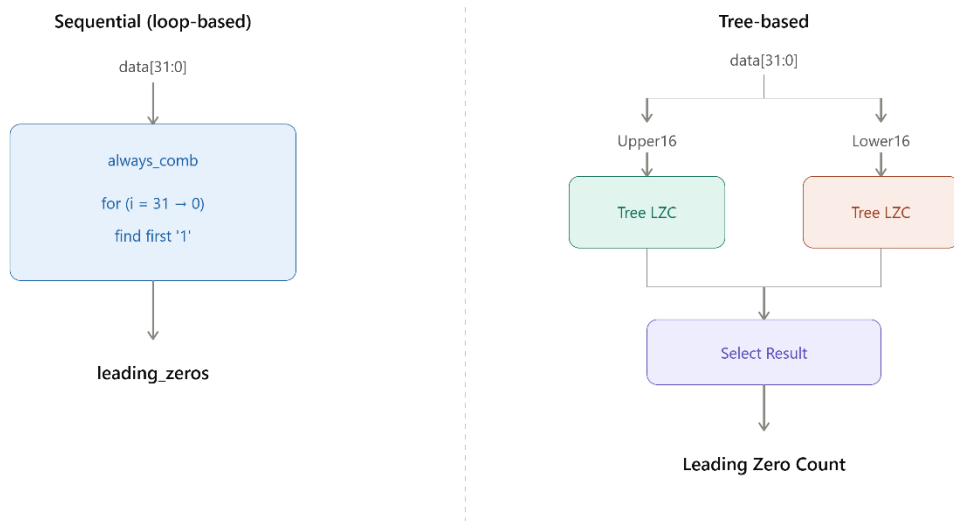
- When sel[2]=0, the lower decoder is enabled and produces outputs y[3:0].
- When sel[2]=1, the upper decoder is enabled and produces outputs y[7:4].

Only one decoder is active at any time, preventing multiple outputs from becoming high simultaneously. The testbench exhaustively verifies all eight input combinations with enable asserted and confirms that all outputs become zero when enable is deasserted.

Task 7: 32-bit Leading Zero Counter (LZC)

1. EDA Playground Link: <https://edaplayground.com/x/gEqU>

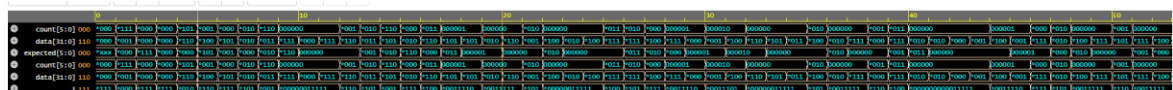
2. Block Diagram:



3. Simulation Output Screenshot:

```
Compiler version X-2025.06-SF1_Full164; Runtime version X-2025.06-SF1_Full164; Jul 20 13:16 2026
[PASS] data=00000000 leading zeros=32
[PASS] data=00000001 leading zeros=31
[PASS] data=00008000 leading zeros=16
[PASS] data=80000000 leading zeros=0
[PASS] data=05e23536 leading zeros=5
[PASS] data=57dea13c leading zeros=1
[PASS] data=be7fe77d leading zeros=0
[PASS] data=30aa33e2 leading zeros=2
[PASS] data=0279440b leading zeros=6
[PASS] data=b9b381df leading zeros=0
[PASS] data=f350a940 leading zeros=0
[PASS] data=d51778f7 leading zeros=0
[PASS] data=6d7e79a6 leading zeros=1
[PASS] data=30d4b41b leading zeros=2
[PASS] data=025b9db5 leading zeros=6
[PASS] data=82e394fa leading zeros=0
[PASS] data=19bc7a4e leading zeros=3
[PASS] data=5a98a215 leading zeros=1
[PASS] data=7985c17d leading zeros=1
[PASS] data=e43a5f72 leading zeros=0
[PASS] data=ffbd4196 leading zeros=0
[PASS] data=217cb731 leading zeros=2
[PASS] data=9df554c4 leading zeros=0
[PASS] data=cac8ccaa leading zeros=0
[PASS] data=f58bc0c4 leading zeros=0
[PASS] data=11ab26cf leading zeros=3
[PASS] data=38646d4f leading zeros=2
[PASS] data=e8478af4 leading zeros=0
[PASS] data=7337dd17 leading zeros=1
[PASS] data=76264a88 leading zeros=1
[PASS] data=2f9fc9f1 leading zeros=2
[PASS] data=2da1e52c leading zeros=2
[PASS] data=94d34dce leading zeros=0
[PASS] data=9bee7505 leading zeros=0
[PASS] data=93fee0cb leading zeros=0
[PASS] data=3af0db8c leading zeros=2
[PASS] data=ac13f71a leading zeros=0
[PASS] data=8ef14f37 leading zeros=0
[PASS] data=4c23c660 leading zeros=1
[PASS] data=1ac5a95f leading zeros=3
[PASS] data=bb91ab7a leading zeros=0
[PASS] data=b1487ba2 leading zeros=0
[PASS] data=8a4b2cf0 leading zeros=0
[PASS] data=f7fb42c9 leading zeros=0
```

4. Waveform Screenshot:



5. Explanation: The Leading Zero Counter determines the number of consecutive zero bits starting from the most significant bit of a 32-bit input. For example 10000000... contains 0 leading zeros while 00000001 contains 31 leading zeros.

Linear Version: The first implementation scans the input from bit 31 down to bit 0 using a for loop. As soon as the first logic 1 is found, the loop stops and returns the number of leading zeros. Although this design is simple and easy to understand, every bit may need to be checked before producing the output. Therefore its critical path grows approximately linearly with the input width.

Tree Structured Version: The optimized implementation divides the 32-bit input into smaller groups. Instead of checking one bit at a time, multiple groups are examined simultaneously. The search proceeds in stages:

- 32 bits
- 16 bits

8 bits
4 bits
2 bits
1 bit

This significantly reduces the number of logic levels and therefore decreases propagation delay. The tree architecture is commonly used in modern CPUs because it provides much faster operation for large bit widths.

Why is the Tree Version Faster?

The linear version examines one bit after another, meaning every additional bit increases the delay. The tree version checks many bits simultaneously by dividing the input into smaller groups and combining the results hierarchically. Since only a few levels are required, the propagation delay is greatly reduced, making it suitable for high-speed processor datapaths.